What Will Probably Be on the Exam – TWUN13

Comparison of the 3 old exams with your 9 lectures. E1 = WEB25-19-25, E2 = 2026-01-14 (newest), E3 = re-exam 2025-01-09.

1. The most important finding

E1 and E2 have exactly the same structure. Same 10 categories (T1–T10), same number of questions per category, same points. Only the questions inside change. So your exam will very likely look like this: *(Tenta 1 och 2 är byggda på exakt samma mall – din tenta följer troligen den.)*

Part	Category	Questions	Points	Lectures
T1	General (web app, client–server, HTTP stateless)	3	3	L07
T2	HTML / CSS / JS	5	5	L01, L02, L03
T3	Front-end / React	5	5	L04, L05
T4	Back-end / Node.js / Express	5	5	L07
T5	Browser–server communication (HTTP, fetch, CORS)	4	4	L07 (+ missing)
T6	Database (MongoDB, NoSQL, cookies)	3	3	L09
T7	Security (STRIDE, SQL injection)	2	2	**not in your files**
T8	Performance (CDN, caching, load balancing)	2	2	**not in your files**
T9	State management (Context, state updates)	2	2	L04, L06a
T10	**Code comprehension (write the answer + explain)**	**4**	**19–22**	**L02, L03, L05**
	Total	**35**	**50**	

> **Remember: T10 alone = about 40% of the exam** (19–22 points). It is almost only useState / useEffect / hoisting. If you master those, you are close to a pass (20 points) from T10 + a few easy multiple-choice.

2. Points per topic on each exam

Topic	E1	E2	E3	Avg	Priority
React (state, useEffect, render, Context, Router)	22	23	11	**19**	★★★ #1
Node.js / Express / middleware	5	4	10	6	★★★ #2
HTTP, web basics, URL, status codes, fetch, CORS	6	6	7	6	★★★ #3
JS core (==, +, hoisting, scope, functions)	6	5	2	4	★★★ #4
HTML / CSS / DOM / events	2	3	6	4	★★
Async / event loop / Promises	2	1	3	2	★★
Databases (MongoDB, NoSQL, Mongoose)	3	2	3	3	★★
Cookies / sessions / auth	0	1	3	1–2	★★
Security (STRIDE, injection, HTTPS)	2	2	2	2	★★ (easy points)
Performance (CDN, cache, load balancing)	2	2	2	2	★★ (easy points)
TypeScript	0	0	1	–	★
Your own project (E3 only)	0	0	2	–	★

(E2 Q3 was an empty question, 1 point. E3 had 52 points with the optional project question.)

3. Questions that came back on several exams

These are the safest bets. The same idea was asked 2–5 times. *(Samma idé har kommit flera gånger – plugga dessa först.)*

Theme	Where	What to know
setState → infinite re-render	E1 Q33, E2 Q12, E2 Q33	setState in the body = "Too many re-renders" error. useEffect that sets the state it depends on = endless loop.
useEffect dependency array	E1 Q11, E1 Q35, E2 Q11, E2 Q13, E3 Q12	none = every render, [] = once, [x] = when x changes. Cleanup = returned function.
Hoisting with var	E1 Q32 (4p), E2 Q8	Declaration moves up, value doesn't → undefined.
Event loop order	E1 Q3, Q7, Q8, E2 Q17, E3 Q37	sync → Promise → setTimeout. var in loop → 3 3 3. async returns a Promise right away.
Middleware	E1 Q16, Q17, Q18, E2 Q16, E3 Q15	app.use, (req,res,next), runs in order, next() = go on, can end the cycle.
Context	E1 Q12, Q31, E2 Q30, Q31	Solves prop drilling, Provider gives the value, all users re-render on change.
CORS purpose	E1 Q21 = E2 Q21 (identical!)	Lets the browser access APIs on other domains.
fetch + HTTP errors	E1 Q22, E2 Q22	fetch doesn't reject on 404/500 → check res.ok yourself.
Type coercion	E1 Q6, E2 Q32 (3p)	5 == "5" → true. "5" + 1 → "51".
"What does this render?"	E1 Q34, E2 Q34 (5p each)	Simple JSX: && / ternary / {name}. Say the HTML + one reason.
"Explain why…" React	E1 Q35, E2 Q35 (5–6p)	Timer + cleanup / state kept between renders + prev => prev + 1.
Node.js / Express definition	E1 Q14, E2 Q14, E3 Q14	Node = runtime for JS on server. Express = Node framework for web apps/APIs.
MongoDB	E1 Q23–25, E2 Q24–25, E3 Q23–24	NoSQL document DB, collection, _id, schema for query patterns, Mongoose model.
STRIDE	E1 Q27 (R), E2 Q26 (S), Q27 (I), E3 Q27 (D)	One letter per exam. **T (Tampering) and E (Elevation of privilege) have not been asked yet** → good guess for next time.
CDN / caching / load balancing	E1 Q28, E2 Q28–29, E3 Q31	Nearest server / store temp data / spread traffic.
CSS selectors	E1 Q5, E2 Q5	id > class > tag. `div p` vs `div>p` vs `div+p`.
HTTP status codes / methods	E1 Q19, E2 Q19, E3 Q18–20	PUT = replace all. 404 not found, 4xx client, 5xx server.

4. Prediction for T10 – the 4 code questions (19–22 points)

On E1 and E2, the 4 code questions followed the **same pattern**. Expect the same: *(Samma mönster båda gångerna.)*

Question	Pattern on E1 / E2	Most likely next time
Q32 (3–4p) JS basics	E1: hoisting `var x` → undefined. E2: `"5" + 1` → "51".	A JS trap: hoisting, ==/===, string + number, var/let in loop, truthy/falsy, const object change, let → ReferenceError.
Q33 (5p) "What happens when this component renders?"	E1: useEffect sets count with [count] → endless loop. E2: setCount in body → Too many re-renders.	**Again an infinite loop variant**: useEffect with NO array that sets state, fetch in the body, or the correct version with [] (runs once).
Q34 (5p) "What does this render?"	E1: `show && <h1>`. E2: `<p>Hello {name}</p>`.	Simple JSX: ternary `? :`, `&&` with false or **0** (0 is rendered as "0"!), a list with map + key, props passed to a child.
Q35 (5–6p) "Explain why..."	E1: text disappears after 2 s (timer + cleanup). E2: count doesn't reset (state kept, prev => prev + 1).	useEffect cleanup, state kept between renders, **batching** (3× `setCount(count+1)` adds only 1, but 3× `setCount(p => p+1)` adds 3), props read-only.

Remember: How to write a T10 answer (max ~50 words): 1) the output in one line, 2) name the concept (hoisting / re-render / dependency array / cleanup / batching), 3) one sentence on the chain of events. Example: "Too many re-renders error. setCount runs in the body, so every render changes state, which starts a new render → infinite loop."

5. Questions from YOUR lecture slides (very likely to be reused)

Your teachers put quiz questions in the Fall 2026 slides. Teachers often reuse these on the exam. Know all the answers: *(Lärarna lägger ofta sina föreläsningsquiz på tentan.)*

Code (from L02 / L03)	Answer
`4 * "2" ** 3 + 10`	**42** (** first, "2" → 2)
`"100" == 100` / `"100" === 100`	**true** / **false**
`"Hello" && 0`	**0** (&& returns the first falsy value)
`let x; console.log(x);`	**undefined**
`const π = 3.14; π = 22/7;`	**TypeError**: Assignment to constant variable
`const squares = [1,4,9,16]; squares[4] = 25;`	**[1, 4, 9, 16, 25]** (const doesn't lock the content)
`user.hobbies` (property doesn't exist)	**undefined** (no error)
`user.age = undefined; console.log(user)`	**{name: "Lea Verou", age: undefined}** (not deleted)
`hobbies.user = "Lea"; hobbies.length` (array of 2)	**2** "Lea Verou" (arrays are objects)
`hobbies["1"] = "Dining"; hobbies[1]`	**Dining Dining** (index "1" = 1)
`if (true) { let x = 1; } console.log(x);`	**ReferenceError** (let is block-scoped)
`if(true){ let x=1; log(); } function log(){ console.log(x) }`	**ReferenceError** (lexical scope)
`Boolean("0"), Boolean([]), Boolean({})`	**true** (all truthy)
`setTimeout(hello, 0); console.log('I run...'); console.log('... very fast.')`	I run... → ... very fast. → When do I say hello?

Quiz from L05 (React)	Answer
What do props help us do?	Make components **reusable**.
How do you pass a prop?	`<MyHeader title="hi" />`
Can you put `blah={true}` on a `<div>`?	**No** – real DOM elements only have HTML attributes.
What data type is props?	An **object**.
Does changing a normal variable re-render?	**No** – only setState (or new props).

6. New in the Fall 2026 lectures – not asked yet, but could be

The slides were updated for this course round. These parts are new or bigger than before, so a new question here is possible:

- **React Router data mode**: `createBrowserRouter`, `loader` + `useLoaderData()`, `action` + `<Form>`, `<Outlet>`, `useParams()`, `useSearchParams()`, `useNavigate()`, `<NavLink>`.
- **Authentication**: authentication vs authorization, session flow, **hashing + salt**, Passport (`serializeUser`, `req.user`, `req.isAuthenticated()`), cookie attributes **HttpOnly / Secure / SameSite**, XSS vs CSRF, 401 on wrong login.
- **Databases**: **ACID**, **vertical vs horizontal scaling**, the 4 NoSQL types (key-value, column, graph, document), normalized vs denormalized, MongoDB `find()` vs SQL `SELECT`.
- **JS basics**: truthy/falsy, `&&` returns a value, `const` with objects, NodeList vs HTMLCollection, `textContent` vs `innerHTML` (security), `preventDefault`, `stopPropagation`, event delegation, `Promise.all` / `Promise.race`.
- **HTTP**: URL parts + default ports **80 / 443**, the request/response message structure, **201 Created, 204 No Content, 401 Unauthorized**, `req.query` vs `req.params` vs `req.body`.
- **React**: JSX → `React.createElement`, keys in lists, Fragments, Virtual DOM diffing, "lifting state up", the React 19 Context syntax `<MyContext value={...}>`.

7. Gaps: what the exams ask that your files don't cover

> **Trap: Lecture L08 is missing** (your files jump from L07 to L09). Security (STRIDE) and Performance (CDN, caching, load balancing, lazy loading) give **4 points on every exam** but are in none of your 9 files. TypeScript and Mongoose/ORM were on E3. **Get L08 (and any security/performance slides) from the course page.** Until then, the last section of the summary PDF covers what the old exams asked.

8. Study plan by priority

Order	What	Why	Time
1	**useState + useEffect** (L04, L05): the 4 patterns, dependency array, cleanup, batching, re-render rules	~19 points on average, all of T10	40%
2	**Express + HTTP** (L07): middleware, routes, req/res, status codes, methods, message structure, fetch/CORS	~12 points	20%
3	**JS traps + event loop** (L02, L03): hoisting, coercion, scope, sync → Promise → setTimeout	~6 points + T10 Q32	15%
4	**Memorize lists**: STRIDE, CDN/cache/load balancing/lazy loading, MongoDB terms, cookies	~7 easy points	10%
5	Context + Router, HTML/CSS, auth/sessions	~6 points	10%
6	Do all 3 old exams again without looking at the answers	Check yourself	5%

> **Remember:** Pass = **20 points**. A realistic safe plan: get ~12 of the 19–22 T10 points + ~15 of the 31 multiple-choice points = ~27 = grade 3 with margin.